

webGP_v2 — Documentación de Arquitectura y Proyecto

Grupo Petersen — Rediseño web
Stack: Next.js 15 · React 19 · TypeScript · Tailwind CSS 4 · Contentful
Fecha del documento: Julio 2026

Tabla de contenidos

- 1. [Visión general](#)
- 2. [Estructura del repositorio](#)
- 3. [Stack tecnológico](#)
- 4. [Flujo de la aplicación: de la URL al render](#)
- 5. [Integración con Contentful](#)
- 6. [Sistema de bloques CMS](#)
- 7. [Componentes UI \(design system\)](#)
- 8. [Módulo de noticias](#)
- 9. [Layout global \(header y footer\)](#)
- 10. [Caché, ISR y revalidación](#)
- 11. [Convenciones de código](#)
- 12. [Mapa de archivos clave](#)
- 13. [Propuestas de mejora](#)
- 14. [Roadmap sugerido](#)

1. Visión general

webGP_v2 es el rediseño del sitio institucional de **Grupo Petersen**. La arquitectura sigue un patrón **headless CMS**:

- **Contentful** almacena páginas, bloques, noticias, header, footer y entidades de dominio (empresas, sectores, estadísticas, etc.).
- **Next.js App Router** resuelve rutas, obtiene datos en el servidor (RSC) y compone la UI.
- **React Server Components** son la base; los Client Components se reservan para interactividad (navegación móvil, filtros de noticias, carruseles, video con poster).

Principios de diseño

Principio	Implementación
Contenido editable en CMS	Páginas = array de bloques Contentful
Server-first	Fetch en servidor; Suspense para secciones async
Tipado fuerte	Tipos TS por content type en lib/contentful/types/
Separación datos / vista	Resolvers en lib/contentful/{dominio}/ → props de UI
URLs compartibles	Filtros de noticias en query params (?categoria=&empresa=&q=)

2. Estructura del repositorio

```

webGP_v2/
├─ app/                                # Next.js App Router
│  ├─ layout.tsx                      # Layout raíz: fuentes, header, footer, <main>
│  ├─ page.tsx                       # / → redirect a /inicio
│  ├─ loading.tsx                    # Spinner global de transición
│  ├─ not-found.tsx                 # Página 404
│  ├─ globals.css                   # Tokens Tailwind 4, variables CSS
│  ├─ [...page]/                    # Catch-all: todas las páginas CMS + noticias
│  │  ├─ page.tsx                   # Resolución página vs noticia
│  │  └─ loading.tsx
│  └─ api/revalidate/route.ts       # Webhook ISR on-demand
├─ components/
│  ├─ blocks/                       # Bloques CMS + PageRenderer + registry
│  ├─ layout/                       # Header, Footer, navegación
│  ├─ news/                         # UI del módulo de noticias (35 archivos)
│  └─ ui/                           # Design system reutilizable (~49 componentes)
├─ lib/
│  ├─ contentful/                   # Cliente, queries, tipos, resolvers por dominio
│  ├─ news/                         # Mappers y utilidades de noticias
│  ├─ footer/                      # Tipos y constantes del footer
│  ├─ hooks/                       # useDebounceValue (búsqueda)
│  ├─ fonts/                       # Playfair Display
│  ├─ env.ts                       # Variables de entorno
│  └─ utils.ts                     # cn() - clsx + tailwind-merge
├─ config/
│  ├─ site.ts                      # homePath, nombre del sitio
│  └─ layout.ts                    # Tokens de layout (altura header)
├─ features/                        # Reservado (solo .gitkeep; sin módulos aún)
├─ public/                          # Assets estáticos
└─ docs/                           # Documentación del proyecto

```

Directorios vacíos o pendientes

Directorio	Estado
features/	Documentado en README pero sin implementación
components/sections/	Vacío

3. Stack tecnológico

Categoría	Tecnología	Versión
Framework	Next.js (App Router, Turbopack en dev)	15.2.8
UI	React	19

Lenguaje	TypeScript	5
Estilos	Tailwind CSS	4
CMS	contentful SDK	11
Rich text	@contentful/rich-text-react-renderer	16
Utilidades CSS	clsx, tailwind-merge, class-variance-authority	—
Fechas	dayjs	1.11
Lint	ESLint + eslint-config-next	9

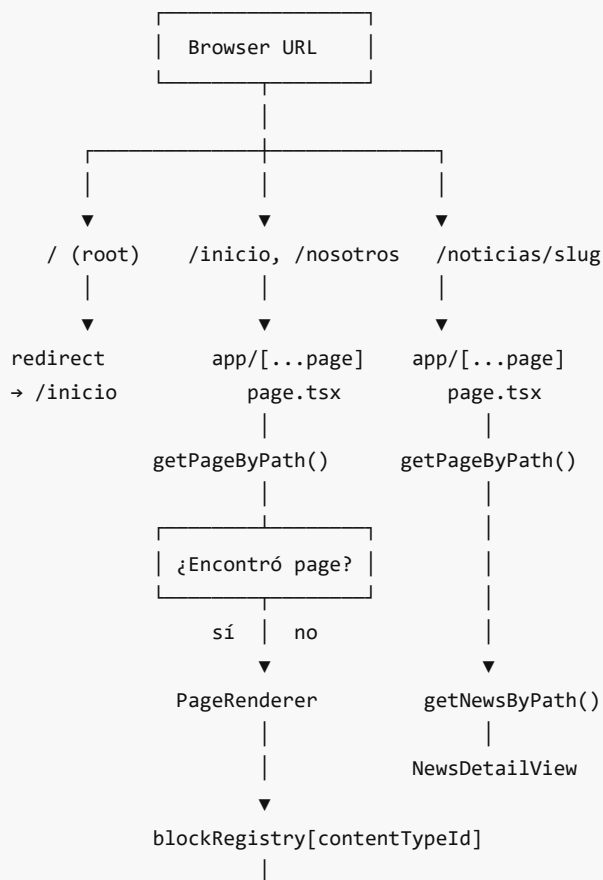
Scripts npm

```

npm run dev      # next dev --turbo
npm run build    # Build de producción
npm run start    # Servidor en puerto 8080
npm run lint     # ESLint

```

4. Flujo de la aplicación: de la URL al render



Paso a paso

1. **app/layout.tsx** — En cada request carga el header desde Contentful (`getHeader()`), aplica fuentes (Open Sans + Playfair Display), envuelve `{children}` en `<main>` y renderiza `SiteFooter` .
2. **app/page.tsx** — Redirige `/` → `/inicio` usando `siteConfig.homePath` .
3. **app/[...page]/page.tsx** — Punto central de routing:
 - Convierte segmentos URL en path (`/foo/bar`).
 - Intenta `getPageByPath(page)` → si hay match, renderiza `PageRenderer` con `fields.content` .
 - Si no hay página, intenta `getNewsByPath(page)` → si hay match, renderiza `NewsDetailView` .
 - Si ninguno coincide → `notFound()` .
4. **PageRenderer** — Itera el array de entries CMS:
 - Casos especiales I3 (innovación + conversaciones emparejadas).
 - Lookup en `blockRegistry` por `entry.sys.contentType.sys.id` .
 - Lógica extra para `/noticias` (filtros, sección "Todas las noticias").
5. **Metadata SEO** — `generateMetadata()` en `[...page]/page.tsx` usa campos `title` , `metaDescription` , `keywords` de page o news.

5. Integración con Contentful

Cliente (`lib/contentful/client.ts`)

- Factory `getContentfulClient()` con SDK oficial.
- Alterna Delivery API / Preview API según `lib/env.ts` .

Variables de entorno (`lib/env.ts`)

Variable	Uso
CONTENTFUL_SPACE_ID	ID del space
CONTENTFUL_ACCESS_TOKEN	Token Delivery
CONTENTFUL_PREVIEW_ACCESS_TOKEN	Token Preview (opcional)
CONTENTFUL_USE_PREVIEW	Flag preview
REVALIDATE_SECRET	Secreto webhook ISR
BASE_URL	URL base del sitio

Queries principales (`lib/contentful/queries.ts`)

Función	Content type	Descripción
<code>getPageByPath(page[])</code>	page	Busca por <code>fields.path.fields.link</code> ; include: 10

<code>getNewsByPath(page[])</code>	news	Slug desde <code>/noticias/{slug}</code>
<code>getNewsListItems(options)</code>	news	Listado paginado con filtros
<code>getLatestNews(limit)</code>	news	Wrapper de listado
<code>getRelatedNews(...)</code>	news	Misma categoría, luego fallback
<code>getHeader()</code>	header	Entrada única
<code>getFooter()</code>	footer	Entrada única
<code>getNewsFilterCompanies()</code>	company	Empresas para filtro
<code>getFooterCompanyLinks()</code>	company	Links externos footer
<code>getFooterDownloadableResources()</code>	downloadableDocuments	PDFs/recursos
<code>getFooterPageLinks()</code>	page	Links internos (excluye home)
<code>getAllPagePaths()</code>	page	Definida pero no usada

Tipos Contentful (`lib/contentful/types/`)

22 módulos de tipos exportados desde `index.ts` . Cada content type tiene:

- Interface de fields (`PageFields` , `NewsFields` , etc.)
- Skeleton para el SDK (`PageSkeleton` , etc.)
- Helpers de dominio (`getLinkHref()` , `isXxxSectionContentfulName()` , etc.)

Resolvers por dominio

Los resolvers transforman entries Contentful crudos en props listas para componentes UI:

Carpeta	Responsabilidad
<code>banner/</code>	Items de tarjetas en banners
<code>carousel/</code>	Items de carrusel de empresas
<code>company/</code>	Logos, URLs de sitios web
<code>foundations/</code>	Items de sección Fundaciones
<code>gridSection/</code>	URLs de links externos en secciones
<code>image/</code>	Items de imagen (fundaciones, etc.)
<code>news/</code>	Filtros URL, categorías, estilos badge
<code>richText/</code>	Quotes embebidos en rich text
<code>sector/</code>	Grids de sectores, empresas por sector
<code>statistic/</code>	Items de estadísticas (banners, economías regionales)
<code>strategicSector/</code>	Sectores estratégicos interactivos

video/	URLs embed, videos de sección
--------	-------------------------------

Helpers compartidos

- `getAssetUrl.ts` — Normaliza URLs de assets Contentful
- `getAssetDimensions.ts` — Dimensiones para `next/image`
- `resolveNavLink.ts` — Resuelve entries `link` / `externalLink` a href + label

6. Sistema de bloques CMS

Registry (`components/blocks/registry.ts`)

Mapa `content type ID` → `componente React`:

Content type ID	Componente	Rol
banner	BannerBlock	Heroes: default, estadísticas, Qualia seguros
card	CardBlock	Tarjeta CTA Fundaciones
externalLink	ExternalLinkBlock	Embed iframe de URL externa
featuredNews	FeaturedNewsBlock	Hub de noticias en <code>/noticias</code>
gridSection	GridSectionBlock	Bloque polivalente (~15 variantes)
image	ImageBlock	Imagen única en sección
richTextBlock	RichTextBlock	Texto enriquecido
sectorsGridSection	SectorsGridSectionBlock	Grid de sectores (tipo propio)
video	VideoBlock	Video Contentful

PageRenderer (`components/blocks/PageRenderer.tsx`)

Orquestador principal. Responsabilidades:

1. **Renderizado secuencial** de bloques del CMS.
2. **Emparejamiento I3** — Si dos `gridSection` consecutivos son "Innovación" + "Conversaciones", los fusiona en `I3InnovationSplitSection` .
3. **I3 standalone** — Innovación sola o Conversaciones sola tienen render propio.
4. **Vista filtrada de noticias** — Con filtros activos en URL, solo muestra bloque `featuredNews` del CMS; oculta el resto.
5. **Sección "Todas las noticias"** — En `/noticias` sin filtros, append de `AllNewsSectionWithFetch` dentro de `Suspense`.

GridSectionBlock — variantes por `contentfulName`

El bloque más complejo (~546 líneas). Según el nombre CMS (`contentfulName`), título, estilo de tarjeta y contexto de página, renderiza:

Variante (detección)	Componente UI
----------------------	---------------

Grid de sectores	SectorsGridSection
Grid de logos	LogoGridSection
Noticias relacionadas	YouMayAlsoLikeSection
Empresas del sector	SectorCompaniesSectionWithFetch
CTA Hiring Room	HiringRoomCtaSection
Dinámicas presenciales	InPersonDynamicsSection
Fundaciones GP	FoundationsGridSection
CTA Conoce Fundaciones	FoundationsConoceCtaCard
Compromiso regional / economías	RegionalCommitmentSection
Sectores estratégicos	StrategicSectorsSection
Carrusel empresas	CarouselSectionWithFetch
Programas de desarrollo	ProgramsDevelopmentSection
Tarjetas overlay	ImageOverlayGridSection
Tarjetas con borde	BorderedGridSection
Tarjetas con ícono	IconCardGridSection
Talento y cultura	TalentCultureSection
Default	Layout imagen lateral + rich text + grid de cards

La detección usa funciones `isXxxSectionContentfulName()` en `lib/contentful/types/gridSection.ts`, comparando strings normalizados del CMS.

Bloques auxiliares (no en registry)

Archivo	Uso
<code>CarouselSectionWithFetch.tsx</code>	Server component async — fetch carrusel
<code>SectorCompaniesSectionWithFetch.tsx</code>	Server component — empresas por sector
<code>LogoGridSectionWithFetch.tsx</code>	Sin imports — código muerto

BannerBlock — sub-variantes

- Banner hero estándar → `HeroBanner`
- Banner con estadísticas → `StatisticsBanner` + `resolveStatisticItems`
- Banner Qualia → `QualiaBanner`
- Opcional: rich text + CTAs debajo del hero

7. Componentes UI (design system)

Ubicación: `components/ui/` . Export barrel en `components/ui/index.ts` (~37 componentes).

Primitivos de layout

Componente	Función
Container	Ancho máximo y padding horizontal
Section	Wrapper de sección con spacing vertical
Button	Botón con variantes CVA
Badge	Etiquetas/chips

CMS / media

Componente	Función
ContentfulImage	Imagen optimizada con next/image
ContentfulVideo	Video desde entry Contentful
RichText	Renderer compartido de rich text
ExternalLinkEmbed	iframe para URLs externas
CtaLinks	Lista de CTAs (link / externalLink)
VideoEmbedPoster	Video con poster clickeable (client)
QuoteBlock	Bloque de cita

Heroes y banners

Componente	Función
HeroBanner	Hero principal con imagen/fondo
StatisticsBanner	Hero con métricas numéricas
QualiaBanner	Hero específico Qualia Seguros
NewsHero	Hero del listing de noticias

Grids y secciones de dominio

Componente	Función
GridCard, BorderedGridCard, GridOverlayCard	Tarjetas de grid
BorderedGridSection, IconCardGridSection, ImageOverlayGridSection	Layouts de grid
LogoGridSection, CompanyLogoGrid	Grids de logos
SectorsGridSection, SectorsGridCard	Grid de sectores
SectorCompaniesSection, SectorCompanyCard	Empresas de un sector

StrategicSectorsSection, StrategicSectorsInteractive, StrategicSectorCard, StrategicSectorDetailPanel	Sectores estratégicos interactivos
FoundationsGridSection, FoundationsImageGrid, FoundationsConoceCtaCard	Secciones Fundaciones
RegionalCommitmentSection	Compromiso regional con estadísticas
TalentCultureSection	Talento y cultura
HiringRoomCtaSection	CTA Hiring Room
InPersonDynamicsSection	Dinámicas presenciales
ProgramsDevelopmentSection, ProgramDevelopmentCard	Programas de desarrollo
I3InnovationSplitSection, I3InnovationPanel, I3ConversationsPanel	Sección I3 Innovación
CarouselSection, MarqueeCarousel	Carruseles

Noticias (badges en UI)

Componente	Función
NewsCategoryBadge	Badge de categoría
NewsCompanyBadge	Badge de empresa
icons/NewsMetaIcons.tsx	Íconos meta (fecha, lectura)

Loading

Componente	Función
PageLoadingState	Spinner full-page
LoadingSpinner	Spinner inline

8. Módulo de noticias

Rutas

URL	Vista
/noticias	Listing hub (FeaturedNewsBlock + secciones)
/noticias/{slug}	Artículo individual (NewsDetailView)
/noticias?categoria=X&empresa=Y&q=Z	Listing filtrado

Parámetros URL (lib/contentful/news/newsListFilters.ts)

Param	Significado
-------	-------------

categoria	Categoría editorial o Todo
empresa	Nombres de empresa separados por coma (legibles, no IDs)
q	Búsqueda full-text Contentful

Categorías (lib/contentful/news/newsCategories.ts)

Todo , Sustentabilidad , Innovación , Educación , Desarrollo regional , Comunidad , Institucional , Minería

Componentes de noticias (components/news/)

Listing

Componente	Tipo	Función
FeaturedNewsHero	Server	Hero del hub
FeaturedNewsSection	Server	Grid de destacadas CMS
LatestNewsSection	Server	Últimas noticias
AllNewsSection	Server	Todas las noticias
NewsPageFeaturedGrid	Server	Layout grid destacadas
FeaturedNewsCard, LatestNewsCard, AllNewsCard	Server	Tarjetas individuales

Fetch wrappers (Suspense)

Componente	Función
LatestNewsSectionWithFetch	Fetch + Suspense últimas
AllNewsSectionWithFetch	Fetch + Suspense todas
FilteredNewsSectionWithFetch	Resultados filtrados server-side
NewsFiltersSectionWithFetch	Filtros con datos de empresas

Filtros (client)

Componente	Función
NewsFiltersSection	Orquestador: chips, dropdown, búsqueda
NewsCategoryFilterChip	Chip de categoría con deselect
NewsCompanyFilterDropdown	Multi-select empresas
NewsFilterSheet	Panel filtros mobile

Comportamiento:

- Debounce 400ms en búsqueda (useDebounceValue)
- useTransition para feedback "Actualizando resultados..."

- Estado en URL (shareable, back-button friendly)

Artículo individual

Componente	Función
NewsDetailView	Layout completo del artículo
NewsArticlePageScale	Escala tipográfica 100% vs 90% global
NewsArticleMeta	Fecha, tiempo lectura, empresa
NewsArticleBody	Cuerpo rich text
NewsSidebar, NewsSidebarMeta	Sidebar con meta
NewsShareSection	Compartir
NewsTagsSection	Tags
NewsCompanyInfoSection	Info de empresa
RelatedNewsSection, YouMayAlsoLikeSection	Noticias relacionadas

Mappers (lib/news/)

Módulo	Función
mapFeaturedNewsItem.ts	Entry → props tarjeta destacada
mapNewsListItem.ts	Entry → props tarjeta listado
mapRelatedNewsItem.ts	Entry → props related
mapNewsEntriesFromItems.ts	Items de gridSection → noticias
buildNewsArticleContext.ts	Contexto artículo (fecha, lectura, empresa)
formatNewsDate.ts	Formato de fecha (days)
readingTime.ts	Tiempo de lectura desde rich text
richTextPlainText.ts	Extracción texto plano
truncateText.ts	Truncado de texto

Comportamiento en /noticias

Sin filtros:

- Hero + filtros
- Destacadas CMS
- Últimas noticias
- Todas las noticias (append desde PageRenderer)
- Resto de bloques CMS de la página

Con filtros activos (categoria , empresa o q):

- Hero + filtros
- Grilla filtrada (`FilteredNewsSectionWithFetch`)
- **Ocultos**: destacadas, últimas, todas, otros bloques CMS

9. Layout global (header y footer)

Header (`components/layout/`)

Archivo	Tipo	Función
<code>Header.tsx</code>	Server	Resuelve logo + nav desde Contentful
<code>HeaderNav.tsx</code>	Client	Menú mobile, navegación interactiva

Datos: `getHeader()` en layout raíz.

Footer (`components/layout/`)

Archivo	Función
<code>SiteFooter.tsx</code>	Server — agrega datos de múltiples queries
<code>Footer.tsx</code>	Presentacional
<code>footer/FooterColumn.tsx</code>	Columna
<code>footer/FooterLinkList.tsx</code>	Lista de links
<code>footer/FooterIcons.tsx</code>	Íconos redes

Datos agregados en `SiteFooter` :

- Footer CMS entry
- Links de empresas (`getFooterCompanyLinks`)
- Recursos descargables (`getFooterDownloadableResources`)
- Links de páginas (`getFooterPageLinks`)

10. Caché, ISR y revalidación

Config	Valor	Ubicación
<code>revalidate</code>	3600 (1 hora)	<code>app/layout.tsx</code> , <code>app/[...page]/page.tsx</code>

Webhook on-demand

```
POST /api/revalidate?secret=XXX&path=/ruta
```

Invalida caché ISR para una ruta específica cuando Contentful publica cambios.

Suspense y loading

Archivo	Alcance
---------	---------

app/loading.tsx	Transiciones de ruta globales
app/[...page]/loading.tsx	Idem para catch-all
NewsResultsLoading	Skeleton grilla noticias
Boundaries en PageRenderer / FeaturedNewsBlock	Secciones async de noticias

11. Convenciones de código

Convención	Detalle
Alias @/	Imports desde raíz del proyecto
Block registry	Un componente por content type ID
*WithFetch	Server components async envueltos en Suspense
contentfulName	Identificación de variantes de sección por string CMS
Server-first	'use client' solo donde hay interactividad
Tipado Contentful	Skeleton + Fields por content type
Estilos	Tailwind 4 con tokens en globals.css (@theme)
Utilidad cn()	Combina clsx + tailwind-merge

12. Mapa de archivos clave

Responsabilidad	Archivo
Routing catch-all	app/[...page]/page.tsx
Layout raíz	app/layout.tsx
Orquestación bloques	components/blocks/PageRenderer.tsx
Registry bloques	components/blocks/registry.ts
Bloque polivalente	components/blocks/GridSectionBlock.tsx
Hub noticias	components/blocks/FeaturedNewsBlock.tsx
Queries CMS	lib/contentful/queries.ts
Cliente Contentful	lib/contentful/client.ts
Filtros noticias URL	lib/contentful/news/newsListFilters.ts
Detección variantes grid	lib/contentful/types/gridSection.ts
Webhook ISR	app/api/revalidate/route.ts
Config sitio	config/site.ts

13. Propuestas de mejora

13.1 Arquitectura y mantenibilidad

A. Refactorizar `GridSectionBlock` (prioridad alta)

Problema: ~546 líneas con cadena larga de `if/else`. Difícil de testear, extender y revisar en PRs.

Propuesta: Registry de secciones por `contentfulName`:

```
// lib/contentful/gridSection/sectionRegistry.ts
type SectionResolver = (ctx: GridSectionContext) => ReactNode | null;

export const gridSectionRegistry: Array<{
  match: (fields: GridSectionFields, pagePath?: string[]) => boolean;
  render: SectionResolver;
}> = [ /* ... */ ];
```

Beneficios: cada variante en su propio archivo, tests unitarios por sección, onboarding más simple.

B. Extraer lógica especial de `PageRenderer` (prioridad media)

Mover a módulos dedicados:

- `lib/contentful/gridSection/renderI3Sections.ts` — emparejamiento I3
- `lib/news/newsPageOrchestration.ts` — lógica de `/noticias` (filtros, qué bloques mostrar)

C. Unificar detección de páginas (prioridad baja)

`isNewsListingPage()` está duplicada en `PageRenderer.tsx` y `FeaturedNewsBlock.tsx`. Centralizar en `lib/news/isNewsListingPage.ts`.

D. Adoptar `features/` como prometió el README (prioridad media)

Migrar dominios a módulos cohesivos:

```
features/
  news/
    components/
    lib/
    index.ts
  sectors/
  foundations/
```

Cada feature exporta lo que la app consume; `components/blocks/` queda como capa delgada de adaptación CMS.

13.2 Rendimiento

A. Reducir `include: 10` en queries (prioridad alta)

`INCLUDE_DEPTH = 10` trae árboles de entries muy profundos en cada request. Evaluar profundidad real necesaria por query (p.ej. 3–5 para listados, 10 solo para páginas completas).

B. Activar `generateStaticParams` (prioridad media)

`getAllPagePaths()` ya existe pero no se usa. Pre-renderizar rutas conocidas en build reduce TTFB en producción.

```
// app/[...page]/page.tsx
export async function generateStaticParams() {
  const paths = await getAllPagePaths();
  return paths.map((path) => ({ page: path.split('/').filter(Boolean) }));
}
```

C. Paralelizar fetches independientes (prioridad media)

En `SiteFooter` y páginas con múltiples `*WithFetch`, usar `Promise.all` donde no haya dependencias.

D. Optimizar imágenes (prioridad baja)

Verificar que todos los `ContentfulImage` usen `sizes` apropiados y `priority` solo en above-the-fold (hero, LCP).

E. Streaming más granular (prioridad baja)

Envolver bloques pesados (carrusel, sectores estratégicos) en `Suspense` individual con `skeleton` específico, no solo noticias.

13.3 Calidad de código

A. Eliminar código muerto (prioridad alta)

Archivo / símbolo	Acción
<code>LogoGridSectionWithFetch.tsx</code>	Eliminar o conectar
<code>NewsPageNewsSections.tsx</code>	Eliminar o usar
<code>getAllPagePaths()</code> sin <code>generateStaticParams</code>	Usar o documentar por qué no
<code>components/sections/</code> VACÍO	Eliminar o poblar
<code>features/.gitkeep</code>	Implementar o actualizar README

B. Robustecer matching por `contentfulName` (prioridad alta)

Problema: Strings del CMS con `.includes()` son frágiles; un rename en Contentful rompe secciones.

Propuestas:

1. Campo enum/slug en CMS (`sectionVariant: 'regional-commitment'`) en lugar de parsear `displayName`.
2. Al menos: constantes centralizadas + tests de matching.
3. Log en dev cuando ninguna variante matchea (fallback silencioso hoy).

C. Unificar exports de bloques (prioridad baja)

Algunos usan `export const X: BlockComponent`, otros `export function X`. Elegir un estilo.

D. Mover `prettier-plugin-tailwindcss` a `devDependencies` (prioridad baja)

No es runtime dependency.

E. Sanitizar `.env.example` (prioridad alta — seguridad)

No incluir tokens reales; usar placeholders.

13.4 CMS y modelado de contenido

A. Consolidar tipos de sección (prioridad media)

Hoy hay `gridSection` (polivalente) y `sectorsGridSection` (tipo aparte). Evaluar un solo content type con campo `variant` o entries referenciadas por tipo.

B. CardBlock muy acotado (prioridad baja)

Todo el content type `card` sirve una sola variante; el resto de cards vive en `GridSectionBlock`. Unificar o documentar la decisión.

13.5 Experiencia de desarrollo

A. Actualizar README (prioridad media)

El README dice header/footer "pendiente" pero están implementados. Documentar todos los content types soportados.

B. Tests (prioridad media-alta)

Agregar tests para:

- `newsListFilters.ts` — parse/build URL
- `isXxxSectionContentfulName()` — matching de secciones
- Mappers `lib/news/map*.ts`

Framework sugerido: Vitest + Testing Library.

C. Storybook o catálogo de componentes UI (prioridad baja)

Documentar visualmente el design system en `components/ui/`.

D. Preview mode indicator (prioridad baja)

Cuando `CONTENTFUL_USE_PREVIEW=true`, mostrar banner "Preview" en dev/staging.

13.6 Resumen de impacto

Mejora	Esfuerzo	Impacto
Refactor GridSectionBlock → registry	Alto	Mantenibilidad ★★★★★
Reducir include depth	Bajo	Performance ★★★★★
Eliminar código muerto	Bajo	Claridad ★★★
Campo variant en CMS vs contentfulName	Medio	Robustez ★★★★★
generateStaticParams	Medio	Performance ★★★★★
Tests en filtros y matchers	Medio	Confiabilidad ★★★★★

Modularizar features/	Alto	Escalabilidad ★★★★★
-----------------------	------	---------------------

14. Roadmap sugerido

Fase 1 — Higiene (1–2 sprints)

- ☐ Eliminar código muerto
- ☐ Centralizar helpers duplicados (`isNewsListingPage`)
- ☐ Sanitizar `.env.example`
- ☐ Actualizar README
- ☐ Reducir `INCLUDE_DEPTH` donde sea seguro

Fase 2 — Robustez (2–3 sprints)

- ☐ Tests unitarios: filtros, matchers, mappers
- ☐ Campo `sectionVariant` en CMS (migración gradual)
- ☐ `generateStaticParams` para rutas principales

Fase 3 — Arquitectura (3–4 sprints)

- ☐ Registry de secciones para `GridSectionBlock`
- ☐ Extraer orquestación I3 y noticias de `PageRenderer`
- ☐ Estructura `features/` por dominio

Fase 4 — Polish (continuo)

- ☐ Suspense granular por bloque
- ☐ Storybook / catálogo UI
- ☐ Preview mode banner
- ☐ Auditoría Lighthouse / Core Web Vitals

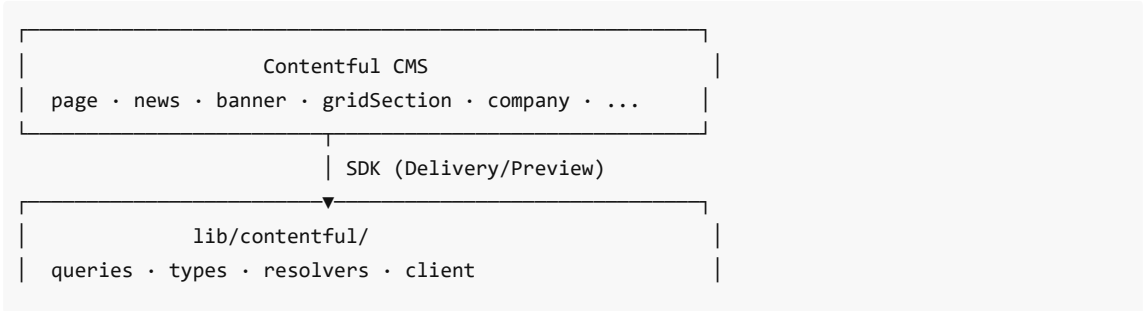
Apéndice A — Content types Contentful registrados

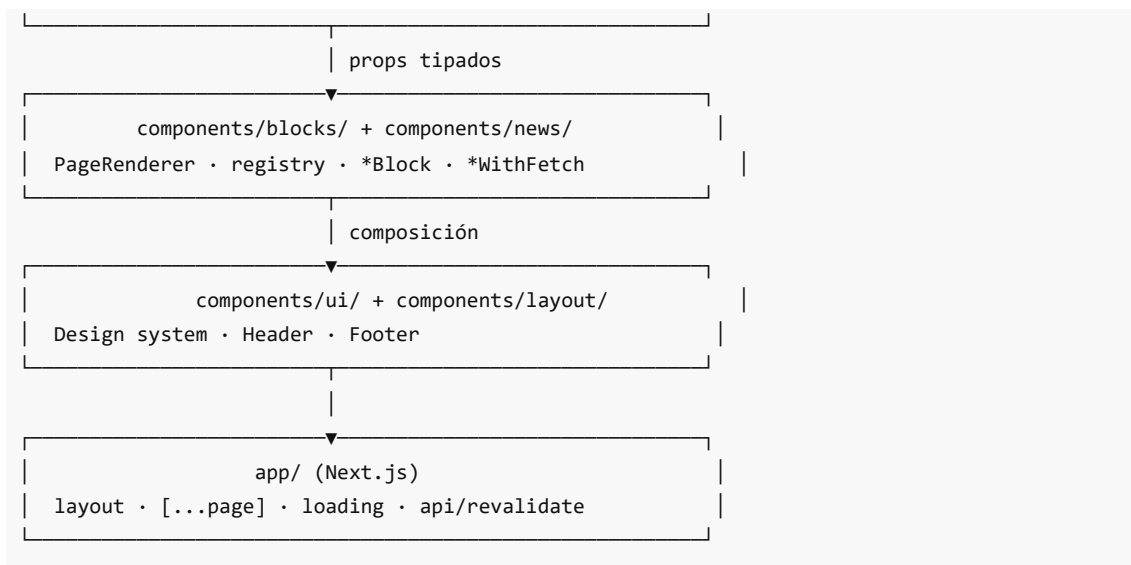
```

banner | card | externalLink | featuredNews | gridSection | image |
sectorsGridSection | richTextBlock | video | page | news | link |
header | footer | company | sector | strategicSector | statistic |
downloadableDocuments | quoteBlock | navigationMenu

```

Apéndice B — Diagrama de capas





Documento generado a partir del análisis del codebase webGP_v2. Para convertir a PDF: abrir este archivo en VS Code/Cursor con extensión "Markdown PDF", o ejecutar `npx md-to-pdf docs/ARQUITECTURA-PROYECTO.md`.